Discrete adjoints for RBF-FD shape optimization in `Macchiato.jl`

Theory and implementation of the manual gradient pipeline

Davide Miotti

2026-06-02 (working document)

Abstract

We document the full discrete adjoint pipeline implemented in `Macchiato.jl` for shape optimization on RBF-FD meshless discretizations of elasticity. Starting from a continuous shape-optimization problem on a parameterized domain, we develop a five-step manual adjoint that decomposes the sensitivity calculation into a forward elasticity solve caching RBF stencils, a single adjoint solve, extraction of sensitivities at the level of weight-matrix entries, a pure-Julia backward pass through the RBF stencil computations, and the assembly of explicit external contributions. Each step has closed-form analytic content and is validated to machine precision against fifth-order central finite differences. We give detailed derivations for each ingredient: the interior elasticity rows for plane stress, the Neumann row family with traction BCs, the shape-dependent dead-load contribution (Phase D Level 1), the differentiable polyline normal model (Phase D Level 2), and the Helmholtz-PDE sensitivity filter on the boundary loop. A 2D cantilever benchmark illustrates the pipeline end-to-end. A plate-with-hole study then motivates reorganizing the optimizer around a Fourier design parameterization and an actively re-meshed cloud: the boundary gradient noise is traced to the geometry sensitivity of the RBF-FD weights ($\partial \mathbf{W}/\partial \boldsymbol{p}$), frequency filtering is shown unable to cure it, and a clean-DOF design space with a two-front (morph plus re-anchored re-mesh) cloud controller takes its place. We close by mapping the open extensions — generalization beyond linear elasticity, integration with the geometry kernel in `WhatsThePoint.jl`, scale-up, and the path to the longer-term multi-physics framework — against established state-of-the-art positioning.

Status of this report. This is a working technical document, written to support both ongoing development and a future engineering-oriented publication. The cantilever example used for illustration is under active development at the time of writing: in particular, an artefact in the tip-shrinkage gradient was found to have been masked by a previous corner-freeze convention, and is being repaired. The *numerical* compliance figures cited in Section **??** should therefore be regarded as illustrative snapshots; the qualitative behaviour they convey (regularization effect, role of differentiable normals) is robust and is what we use them to demonstrate. Note also that the gradient filter of Section **??**, presented there as the regularization cure, is *superseded* by Section **??**: the plate-with-hole study shows frequency filtering cannot separate the broadband weight-sensitivity artifact from the signal, and the cure is the design-space-plus-re-meshing architecture developed there.

Contents

1 Introduction

Shape optimization treats the shape of a physical domain as a design variable and asks for the geometry that extremizes a physical objective subject to physical constraints. Computing the gradient of the objective with respect to the design — the *sensitivity* — is the central computational task, and the dominant strategy for problems with many design degrees of freedom is the *adjoint method*: a single auxiliary linear solve (the "adjoint") replaces many directional forward solves that direct differentiation would require.

`Macchiato.jl` implements this strategy for radial-basis-function finite-difference (RBF-FD) discretizations. RBF-FD is a meshless method: the discretization is a point cloud rather than a connected mesh, and the differential operators (Laplacian, divergence, mixed partial, etc.) are realized as sparse weight matrices built by local RBF stencil fitting. The meshless choice has structural advantages for shape optimization — discretization smoothly tracks geometry deformation, mesh tangling is not a concept, topology change is absorbed naturally — but it puts a premium on *differentiability of the discretization itself*: the weight matrices, the boundary normals, the per-row coefficient blocks must all be differentiable functions of the point positions, and the adjoint must thread the gradient cleanly through every dependence.

The pipeline reported here is a *manual* discrete adjoint: we write each derivative analytically rather than rely on automatic differentiation through the whole computation. The motivation is twofold. First, AD through the full forward solve incurs significant LLVM compilation cost in our Julia setup; an earlier attempt to do this through Mooncake was abandoned after observing five-minute compile times even at $N = 25$ points. Second, and more importantly, the analytic form makes each component inspectable, validateable against finite differences to machine precision, and reusable across physics: the same Step 2 adjoint solve, the same Step 4 backward pass through the RBF stencils, the same Step 3 coefficient-extraction pattern serve every PDE we will eventually plug into the framework.

This document presents the full chain. Section **??** fixes notation. Section **??** specifies the RBF-FD discretization of plane-stress linear elasticity, including the interior assembly, the Dirichlet rows, and the four-coefficient Neumann row family. Section **??** derives the discrete adjoint from first principles, exposing the $\partial \mathbf{A}/\partial \boldsymbol{p}$ and $\partial \boldsymbol{b}/\partial \boldsymbol{p}$ decompositions that drive every later step. Section **??** walks the five-step pipeline in detail, with the elasticity coefficient extraction in Step 3 worked out explicitly. Section **??** treats the differentiable Neumann data (Phase D Levels 1 and 2): shape-dependent traction values and shape-dependent boundary normals. Section **??** treats sensitivity filtering on the boundary loop, both the simple 3-point average and the Helmholtz-PDE filter. Section **??** describes the verification methodology. Section **??** runs the pipeline on a 2D cantilever example. Section **??** reorganizes the optimizer around a Fourier design space and active re-meshing for the plate-with-hole problem. Section **??** maps the extensions — to other elliptic PDEs, to nonlinear systems, to multi-physics, and to 3D. Section **??** positions the work relative to the state of the art. Section **??** lists open work.

2 Continuous problem statement

2.1 Domain and design parameterization

Let $\Omega(\boldsymbol{l}) \subset \mathbb{R}^d$ ($d \in \{2, 3\}$) be a bounded domain whose shape is determined by a design vector $\boldsymbol{l} \in \mathbb{R}^{n_l}$. The boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$ partitions into disjoint parts where Dirichlet and Neumann data are prescribed. The shape dependence enters through any of three layers, which we keep separate throughout this document because they enter the gradient through distinct paths:

(i) *Boundary-node positions* $\boldsymbol{p}_b(\boldsymbol{l})$ — the discrete-level identity of the boundary.

(ii) *Interior-node positions* $\boldsymbol{p}_i(\boldsymbol{l})$ — generated from $\boldsymbol{p}_b$ by a node-distribution policy (see Section **??**).

(iii) *Boundary normals and traction data* $\boldsymbol{n}(\boldsymbol{l})$, $\boldsymbol{t}(\boldsymbol{l})$ — explicit functions of the boundary geometry whenever the loading is shape-coupled (e.g. parabolic shear on a free end whose length is a design parameter, or follower pressure).

Throughout the body of this report we lump the layered design parameterization into a single flat point-coordinate vector $\boldsymbol{p} \in \mathbb{R}^{dN}$ (the concatenation of all node coordinates), and ask for sensitivities with respect to $\boldsymbol{p}$. The $\boldsymbol{p} \mapsto \boldsymbol{l}$ step is handled by the geometry kernel in `WhatsThePoint.jl` and is, by deliberate design, kept outside the PDE side of the gradient.

2.2 Linear elasticity

The strong form, for plane-stress isotropic linear elasticity in 2D with Lamé parameters $(\lambda^\star, \mu)$:

$$\nabla\cdot\boldsymbol{\sigma}(\boldsymbol{u}) = \mathbf{0} \qquad \text{in } \Omega, \tag{1}$$

$$\boldsymbol{u} = \boldsymbol{u}_D \qquad \text{on } \Gamma_D, \tag{2}$$

$$\boldsymbol{\sigma}(\boldsymbol{u})\cdot\boldsymbol{n} = \boldsymbol{t} \qquad \text{on } \Gamma_N, \tag{3}$$

with the constitutive law $\boldsymbol{\sigma}(\boldsymbol{u}) = \lambda^\star(\nabla\cdot\boldsymbol{u})\,\mathbf{I} + \mu(\nabla\boldsymbol{u} + \nabla\boldsymbol{u}^\top)$ and outward unit normal $\boldsymbol{n}$. The displacement $\boldsymbol{u} : \Omega \to \mathbb{R}^2$ is the state.

2.3 The shape-optimization problem

Given an objective functional $\mathcal{L}(\boldsymbol{u}, \boldsymbol{p})$ and an optional geometric constraint $V(\boldsymbol{p}) = V_0$ (typically area or volume), the shape-optimization problem is

$$\min_{\boldsymbol{p}} \; \mathcal{L}\big(\boldsymbol{u}(\boldsymbol{p}),\, \boldsymbol{p}\big) \quad \text{s.t.} \quad \begin{cases} \nabla\cdot\boldsymbol{\sigma}(\boldsymbol{u}) = \mathbf{0} & \text{in } \Omega(\boldsymbol{p}), \\ \boldsymbol{u} = \boldsymbol{u}_D & \text{on } \Gamma_D, \\ \boldsymbol{\sigma}\cdot\boldsymbol{n} = \boldsymbol{t} & \text{on } \Gamma_N, \\ V(\boldsymbol{p}) = V_0. \end{cases} \tag{4}$$

For the illustrative problem of Section **??** we use the compliance loss $\mathcal{L}(\boldsymbol{u}, \boldsymbol{p}) = \boldsymbol{b}(\boldsymbol{p})^\top\boldsymbol{u}$ with the area constraint applied as a two-sided quadratic penalty $\rho_{\text{pen}}\big(V(\boldsymbol{p}) - V_0\big)^2$ added to the objective. The treatment of the constraint is otherwise orthogonal to the adjoint machinery developed here.

3 RBF-FD discretization of elasticity

3.1 Stencils and weight matrices

We sample the domain with N collocation points $\{\boldsymbol{p}_i\}_{i=1}^N \subset \overline{\Omega}$ and, for each one, build a local neighborhood ("stencil") of the k nearest neighbors. On each stencil we fit a polyharmonic-spline radial basis $\phi(r) = r^m$ (we use $m = 3$) augmented by polynomials up to degree p (we use $p = 3$), and we solve a small saddle-point system that returns the coefficients of any linear differential operator $\mathcal{L}$ acting on the stencil basis. The resulting weights form a row of the sparse *weight matrix* $\mathbf{W}^{\mathcal{L}} \in \mathbb{R}^{N\times N}$: for each operator we need (second partials $\partial_{xx}, \partial_{yy}, \partial_{xy}$ for elasticity interior assembly; first partials ∂_x, ∂_y for Neumann traction assembly), the same stencil layout produces a different set of weights, but the sparsity pattern is shared across operators on the same stencil.

The action $\mathbf{W}^{\mathcal{L}}$ on a discrete field $\boldsymbol{v} \in \mathbb{R}^N$ approximates $\mathcal{L}v$ at each sample point:

$$(\mathbf{W}^{\mathcal{L}}\boldsymbol{v})_i \;\approx\; (\mathcal{L}v)(\boldsymbol{p}_i), \qquad i = 1, \ldots, N. \tag{5}$$

Two facts will be central to the adjoint construction: *(i)* $\mathbf{W}^{\mathcal{L}}$ is a smooth function of all $\boldsymbol{p}_j$ in the stencil of row i, hence of $\boldsymbol{p}$ globally; *(ii)* the backward pass $\partial\mathbf{W}^{\mathcal{L}}/\partial\boldsymbol{p}$ for given gradient $\Delta\mathbf{W}^{\mathcal{L}}$ is implemented as a pure-Julia routine `_pullback_weights!` in `RadialBasisFunctions.jl` that reuses the saddle-point factorization cached during the forward weight build, costing only $O(k^3)$ per row.

3.2 Block assembly of plane-stress elasticity

The displacement state is interleaved as $\boldsymbol{u} = (u_1, \ldots, u_N, v_1, \ldots, v_N)^\top \in \mathbb{R}^{2N}$. The Navier equations $\nabla\cdot\boldsymbol{\sigma} = \mathbf{0}$ in plane stress become, after expanding the constitutive law,

$$c_1\partial_{xx}u + c_2\partial_{yy}u + c_3\partial_{xy}v \;=\; 0, \tag{6}$$
$$c_3\partial_{xy}u + c_2\partial_{xx}v + c_1\partial_{yy}v \;=\; 0, \tag{7}$$

with $c_1 = \lambda^\star + 2\mu$, $c_2 = \mu$, $c_3 = \lambda^\star + \mu$. Substituting the discrete operators gives the $2N \times 2N$ block matrix

$$\mathbf{A}_{\text{int}} = \begin{bmatrix} c_1\mathbf{W}^{\partial_{xx}} + c_2\mathbf{W}^{\partial_{yy}} & c_3\mathbf{W}^{\partial_{xy}} \\ c_3\mathbf{W}^{\partial_{xy}} & c_2\mathbf{W}^{\partial_{xx}} + c_1\mathbf{W}^{\partial_{yy}} \end{bmatrix}. \tag{8}$$

This block is applied at all interior rows. Boundary rows (Dirichlet, Neumann) overwrite the corresponding rows of $\mathbf{A}_{\text{int}}$ after assembly.

3.3 Dirichlet rows

For each Dirichlet point with global index $i \in \Gamma_D$, the x-equation row i and the y-equation row $i + N$ are overwritten:

$$\mathbf{A}[i,:] = \boldsymbol{e}_i^\top, \quad \mathbf{A}[i{+}N,:] = \boldsymbol{e}_{i+N}^\top, \quad b_i = u^x_{D,i}, \quad b_{i+N} = u^y_{D,i}. \tag{9}$$

These rows do not depend on $\boldsymbol{p}$ at all, so they contribute zero to $\partial\mathbf{A}/\partial\boldsymbol{p}$ and $\partial\boldsymbol{b}/\partial\boldsymbol{p}$.

3.4 Neumann (traction) rows

For each Neumann point with local index $i_{\text{loc}} \in \{1, \ldots, n_{\text{neu}}\}$ and global index $g = \text{neumann\_ids}[i_{\text{loc}}]$, with outward unit normal $\boldsymbol{n} = (n_x, n_y)$, stencil $\mathcal{S}$, and prescribed traction $\boldsymbol{t} = (t_x, t_y)$, the discrete equations $\boldsymbol{\sigma}\cdot\boldsymbol{n} = \boldsymbol{t}$ become, after expanding the constitutive law and applying $\mathbf{W}^{\partial_x}, \mathbf{W}^{\partial_y}$ on the stencil:

$$\begin{aligned}
\mathbf{A}[g,\, j] \;&=\; n_x(\lambda^\star{+}2\mu)\mathbf{W}^{\partial_x}[i_{\text{loc}}, j] + n_y\mu\,\mathbf{W}^{\partial_y}[i_{\text{loc}}, j], \\
\mathbf{A}[g,\, j{+}N] \;&=\; n_y\mu\,\mathbf{W}^{\partial_x}[i_{\text{loc}}, j] + n_x\lambda^\star\mathbf{W}^{\partial_y}[i_{\text{loc}}, j], \\
\mathbf{A}[g{+}N,\, j] \;&=\; n_y\lambda^\star\mathbf{W}^{\partial_x}[i_{\text{loc}}, j] + n_x\mu\,\mathbf{W}^{\partial_y}[i_{\text{loc}}, j], \\
\mathbf{A}[g{+}N,\, j{+}N] \;&=\; n_x\mu\,\mathbf{W}^{\partial_x}[i_{\text{loc}}, j] + n_y(\lambda^\star{+}2\mu)\mathbf{W}^{\partial_y}[i_{\text{loc}}, j],
\end{aligned} \tag{10}$$

for each $j \in \mathcal{S}$, with $b_g = t_x$ and $b_{g+N} = t_y$. The pattern is: each Neumann row entry is a linear combination of one or more weight-matrix entries, with coefficients that depend on $(n_x, n_y, \lambda^\star, \mu)$. We package this in a `TractionLayout` struct, storing for each entry the pair (c_x, c_y) that multiplies $\big(\mathbf{W}^{\partial_x}[i_{\text{loc}}, j], \mathbf{W}^{\partial_y}[i_{\text{loc}}, j]\big)$.

3.5 The full system

After applying (**??**), (**??**), and (**??**), the assembled system is

$$\mathbf{A}(\boldsymbol{p})\,\boldsymbol{u} \;=\; \boldsymbol{b}(\boldsymbol{p}), \tag{11}$$

solved as a single LU factorization for the forward pass. Both $\mathbf{A}$ and $\boldsymbol{b}$ depend on $\boldsymbol{p}$:

- $\mathbf{A}$ via the interior weights $\mathbf{W}^{\partial_{xx}}, \mathbf{W}^{\partial_{yy}}, \mathbf{W}^{\partial_{xy}}$ (always);
- $\mathbf{A}$ via the Neumann weights $\mathbf{W}^{\partial_x}, \mathbf{W}^{\partial_y}$ (whenever Γ_N is non-empty);
- $\mathbf{A}$ via the boundary normals $\boldsymbol{n}$ (whenever the normals are computed from the geometry rather than prescribed — Phase D Level 2);
- $\boldsymbol{b}$ via the traction values $\boldsymbol{t}$ (whenever the loading is shape-coupled — Phase D Level 1);
- $\boldsymbol{b}$ via interior body forces (none in the elasticity examples here, but the framework is set up for it).

The Dirichlet rows of $\mathbf{A}$ and $\boldsymbol{b}$ are independent of $\boldsymbol{p}$ by construction.

4 The discrete adjoint: derivation

4.1 Total derivative of the loss

The loss has the structure

$$\mathcal{L}(\boldsymbol{u}, \boldsymbol{p}) = \mathcal{L}_{\text{state}}(\boldsymbol{u}, \boldsymbol{p}) + \mathcal{L}_{\text{geom}}(\boldsymbol{p}), \tag{12}$$

where $\mathcal{L}_{\text{state}}$ depends on the state $\boldsymbol{u}$ and optionally on $\boldsymbol{p}$ directly, and $\mathcal{L}_{\text{geom}}$ depends on $\boldsymbol{p}$ alone (e.g. the area penalty). The total derivative with respect to $\boldsymbol{p}$ is

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{p}} \;=\; \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}}\,\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}\boldsymbol{p}}}_{\text{state-mediated}} \;+\; \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{p}}}_{\text{explicit}}. \tag{13}$$

The first term is what the adjoint method computes efficiently. The second is assembled explicitly in Step 5 (Section **??**).

4.2 Sensitivity of the state via the implicit function theorem

Differentiating the linear system (**??**) with respect to $\boldsymbol{p}$:

$$\frac{\partial \mathbf{A}}{\partial \boldsymbol{p}}\,\boldsymbol{u} + \mathbf{A}\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}\boldsymbol{p}} \;=\; \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}}, \tag{14}$$

so

$$\frac{\mathrm{d}\boldsymbol{u}}{\mathrm{d}\boldsymbol{p}} \;=\; \mathbf{A}^{-1}\Big(\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} - \frac{\partial \mathbf{A}}{\partial \boldsymbol{p}}\,\boldsymbol{u}\Big). \tag{15}$$

Substituting back into (**??**):

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{p}} \;=\; \frac{\partial \mathcal{L}}{\partial \boldsymbol{u}}\,\mathbf{A}^{-1}\Big(\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} - \frac{\partial \mathbf{A}}{\partial \boldsymbol{p}}\,\boldsymbol{u}\Big) + \frac{\partial \mathcal{L}}{\partial \boldsymbol{p}}. \tag{16}$$

4.3 The adjoint variable

Naively this requires one linear solve per coordinate — $O(dN)$ solves — because $\partial \boldsymbol{b}/\partial \boldsymbol{p}$ and $\partial \mathbf{A}/\partial \boldsymbol{p}$ have a column per coordinate. The adjoint trick is to take the left contraction first. Define

$$\boxed{\boldsymbol{\eta} \;\equiv\; \mathbf{A}^{-\top}\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}}\right)^{\top},} \qquad \text{equivalently } \mathbf{A}^{\top}\boldsymbol{\eta} = (\partial \mathcal{L}/\partial \boldsymbol{u})^{\top}, \tag{17}$$

which costs a single linear solve. Then $(\partial \mathcal{L}/\partial \boldsymbol{u})\,\mathbf{A}^{-1} = \boldsymbol{\eta}^{\top}$, and

$$\boxed{\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{p}} \;=\; \boldsymbol{\eta}^{\top}\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} - \boldsymbol{\eta}^{\top}\frac{\partial \mathbf{A}}{\partial \boldsymbol{p}}\,\boldsymbol{u} + \frac{\partial \mathcal{L}}{\partial \boldsymbol{p}}.} \tag{18}$$

This is the central identity. Every contribution to the gradient is some piece of $\partial \mathbf{A}/\partial \boldsymbol{p}$ or $\partial \boldsymbol{b}/\partial \boldsymbol{p}$ contracted against $\boldsymbol{\eta}$ and $\boldsymbol{u}$, plus the explicit term.

4.4 Decomposition of $\partial \mathbf{A}/\partial \boldsymbol{p}$

The assembled $\mathbf{A}$ depends on $\boldsymbol{p}$ through several intermediate quantities. We unpack the chain rule, organizing the result by which component of the assembly responds:

$$\frac{\partial \mathbf{A}}{\partial \boldsymbol{p}} \;=\; \sum_{\mathcal{L}\in\{\partial_{xx},\partial_{yy},\partial_{xy},\partial_x,\partial_y\}} \frac{\partial \mathbf{A}}{\partial \mathbf{W}^{\mathcal{L}}}\frac{\partial \mathbf{W}^{\mathcal{L}}}{\partial \boldsymbol{p}} \;+\; \frac{\partial \mathbf{A}}{\partial \boldsymbol{n}}\frac{\partial \boldsymbol{n}}{\partial \boldsymbol{p}}. \tag{19}$$

The first sum (over weight matrices) is the part that "every PDE has": it captures how the operators change as the points move. The second term (over normals) is specific to BCs that re-orient under deformation; it is the Phase D Level 2 contribution and is zero whenever the normals are frozen.

4.5 Decomposition of $\partial \boldsymbol{b}/\partial \boldsymbol{p}$

Symmetrically, $\boldsymbol{b}$ depends on $\boldsymbol{p}$ through traction values on Neumann rows and (in principle) through body forces on interior rows:

$$\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} \;=\; \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{t}}\frac{\partial \boldsymbol{t}}{\partial \boldsymbol{p}} \;+\; \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{f}}\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{p}}. \tag{20}$$

The first term is Phase D Level 1 (shape-dependent dead loads). The second is identically zero in our cantilever example (body-force-free) but the framework accommodates it.

4.6 The five computational steps

Combining (**??**) with the decompositions (**??**) and (**??**) yields the explicit form

$$\begin{aligned}\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{p}} \;=\; &\underbrace{\boldsymbol{\eta}^{\top}\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{t}}\frac{\partial \boldsymbol{t}}{\partial \boldsymbol{p}} \;+\; \boldsymbol{\eta}^{\top}\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{f}}\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{p}}}_{\text{Step 5b: } \partial \boldsymbol{b}/\partial \boldsymbol{p}} \\ &- \underbrace{\sum_{\mathcal{L}} \boldsymbol{\eta}^{\top}\frac{\partial \mathbf{A}}{\partial \mathbf{W}^{\mathcal{L}}}\frac{\partial \mathbf{W}^{\mathcal{L}}}{\partial \boldsymbol{p}}\,\boldsymbol{u}}_{\text{Steps 3+4: } \partial \mathbf{W}/\partial \boldsymbol{p}} \;-\; \underbrace{\boldsymbol{\eta}^{\top}\frac{\partial \mathbf{A}}{\partial \boldsymbol{n}}\frac{\partial \boldsymbol{n}}{\partial \boldsymbol{p}}\,\boldsymbol{u}}_{\text{Step 5c: } \partial \boldsymbol{n}/\partial \boldsymbol{p}} \;+\; \underbrace{\frac{\partial \mathcal{L}}{\partial \boldsymbol{p}}}_{\text{Step 5a: explicit}}.\end{aligned} \tag{21}$$

This decomposes into the five computational steps that the implementation organizes:

1. **Forward solve**: $\boldsymbol{u} = \mathbf{A}^{-1}\boldsymbol{b}$, caching weight matrices and their RBF stencil factorizations.

2. **Adjoint solve**: $\mathbf{A}^\top \boldsymbol{\eta} = (\partial \mathcal{L}/\partial \boldsymbol{u})^\top$, single linear solve.

3. **Extract $\Delta\mathbf{W}$**: convert $-\boldsymbol{\eta}^\top(\partial \mathbf{A}/\partial \mathbf{W}^{\mathcal{L}})\boldsymbol{u}$ into a per-entry $\Delta\mathbf{W}^{\mathcal{L}}$ for each operator, treating sparsity and per-row coefficients analytically.

4. **Backward through RBF stencils**: convert each $\Delta\mathbf{W}^{\mathcal{L}}$ into a contribution to $\Delta\boldsymbol{p}$ by calling `_pullback_weights!`.

5. **External contributions**: add the explicit $\partial\mathcal{L}/\partial\boldsymbol{p}$ (Step 5a), the $\partial\boldsymbol{b}/\partial\boldsymbol{p}$ piece (Step 5b), and the Phase D Level 2 $\partial\boldsymbol{n}/\partial\boldsymbol{p}$ piece (Step 5c).

We now detail each step.

5 The five-step pipeline in detail

5.1 Step 1 — forward solve and caching

Given $\boldsymbol{p}$, build the five RBF weight matrices needed for elasticity ($\mathbf{W}^{\partial_{xx}}, \mathbf{W}^{\partial_{yy}}, \mathbf{W}^{\partial_{xy}}$ on all points; $\mathbf{W}^{\partial_x}, \mathbf{W}^{\partial_y}$ on the Neumann sub-cloud). For each weight matrix, also retain the per-row *cache* returned by `_build_weights_and_cache` — this is the saddle-point factorization used to produce the row, and it is what `_pullback_weights!` reuses for the backward pass.

Assemble $\mathbf{A}$ by combining the interior block (**??**) with Dirichlet identity rows (**??**) and Neumann rows (**??**). Assemble $\boldsymbol{b}$ from prescribed Dirichlet values, traction values $\boldsymbol{t}$, and (if present) body forces.

Factor $\mathbf{A}$ once (LU) and solve $\boldsymbol{u} = \mathbf{A}^{-1}\boldsymbol{b}$. The same factorization will be reused for Step 2.

5.2 Step 2 — adjoint solve

Form $(\partial\mathcal{L}/\partial\boldsymbol{u})^\top$ analytically (cheap for the typical losses — $2\boldsymbol{u}$ for $\mathcal{L} = \|\boldsymbol{u}\|^2$, $\boldsymbol{b}$ for compliance $\mathcal{L} = \boldsymbol{b}^\top\boldsymbol{u}$, etc.). Solve $\mathbf{A}^\top\boldsymbol{\eta} = (\partial\mathcal{L}/\partial\boldsymbol{u})^\top$ using the cached factorization of $\mathbf{A}$. Cost: one back-substitution.

For compliance specifically, $\boldsymbol{\eta} = \mathbf{A}^{-\top}\boldsymbol{b} = \boldsymbol{u}$ if $\mathbf{A}$ is symmetric — the so-called *self-adjoint* case. Our $\mathbf{A}$ is symmetric for the standard elasticity row family (it can be made so by symmetrizing the Neumann rows), but we run the explicit adjoint solve anyway to avoid relying on this property when the formulation is extended.

5.3 Step 3 — extract $\Delta\mathbf{W}$ from $\Delta\mathbf{A}$

This is the most code-dense step. The goal is to translate

$$\Delta\mathbf{A} \;=\; -\boldsymbol{\eta}\,\boldsymbol{u}^\top \quad \in \mathbb{R}^{2N\times 2N} \tag{22}$$

into per-operator sensitivities $\Delta\mathbf{W}^{\mathcal{L}} \in \mathbb{R}^{N\times N}$ that capture $-\boldsymbol{\eta}^\top(\partial\mathbf{A}/\partial\mathbf{W}^{\mathcal{L}})\boldsymbol{u}$ in a single expression per nonzero entry. The decomposition is done analytically using the elasticity coefficient structure in (**??**) (interior) and (**??**) (Neumann).

5.3.1 Interior rows

Each interior row family is governed by (**??**). For a pair of points (i, j) with j in the stencil of i, the four 2×2 block entries are

$$\mathbf{A}[i,j] = c_1\mathbf{W}^{\partial_{xx}}_{ij} + c_2\mathbf{W}^{\partial_{yy}}_{ij}, \qquad \mathbf{A}[i,j{+}N] = c_3\mathbf{W}^{\partial_{xy}}_{ij}, \tag{23}$$

$$\mathbf{A}[i{+}N,j] = c_3\mathbf{W}^{\partial_{xy}}_{ij}, \qquad \mathbf{A}[i{+}N,j{+}N] = c_2\mathbf{W}^{\partial_{xx}}_{ij} + c_1\mathbf{W}^{\partial_{yy}}_{ij}. \tag{24}$$

Setting $t_{11} = -\eta_i u_j$, $t_{22} = -\eta_{i+N} u_{j+N}$, $t_{12} = -\eta_i u_{j+N}$, $t_{21} = -\eta_{i+N} u_j$ (the four 2×2 entries of $\Delta\mathbf{A}[i, j;\, i+N, j+N]$), the back-projection onto the operators is

$$\boxed{\begin{aligned} \Delta\mathbf{W}_{ij}^{\partial_{xx}} &= c_1 t_{11} + c_2 t_{22}, \\ \Delta\mathbf{W}_{ij}^{\partial_{yy}} &= c_2 t_{11} + c_1 t_{22}, \\ \Delta\mathbf{W}_{ij}^{\partial_{xy}} &= c_3 (t_{12} + t_{21}). \end{aligned}} \tag{25}$$

This is the `extract_weight_sensitivities_elasticity!` routine in `src/optimization/manual_adjoint.jl`. It walks the shared sparsity pattern of the three weight matrices once and populates all three $\Delta\mathbf{W}$ in a single pass.

Per-DOF active mask. At rows that are overwritten with Dirichlet identity (where $\partial\mathbf{A}/\partial\mathbf{W} = 0$ for that row), the contribution must be gated to zero so we do not accidentally pick up the identity-row sensitivities. We carry an `active::Vector{Bool}` of length $2N$ and apply the gating per-DOF:

$$t_{ab} \;=\; \begin{cases} (\text{formula above}) & \text{if active}[i\,(+N \text{ if } a{=}2)] \\ 0 & \text{otherwise.} \end{cases} \tag{26}$$

The per-DOF granularity (not per-point) matters: a corner can be Dirichlet in x but Neumann in y, and the mask correctly suppresses only the Dirichlet half.

Skipping Neumann rows in the interior loop. The interior block (**??**) is not the equation actually imposed on Neumann rows after overwriting in (**??**). The Step 3 loop therefore restricts the interior extraction to rows where the interior assembly is in effect (i.e. rows not in Γ_D and not in Γ_N), via an `interior_rows::BitVector` parameter.

5.3.2 Neumann rows

For each Neumann point with local index i_{loc} and global index g, the Neumann row family (**??**) writes each $\mathbf{A}[g, \cdot]$ and $\mathbf{A}[g{+}N, \cdot]$ entry as a linear combination of $\mathbf{W}^{\partial_x}[i_{\text{loc}}, \cdot]$ and $\mathbf{W}^{\partial_y}[i_{\text{loc}}, \cdot]$ with coefficients that depend on $(n_x, n_y, \lambda^\star, \mu)$. Storing these coefficients in a flat array `coeffs`, the `TractionLayout` struct catalogues, for each entry, its (c_x, c_y) pair and the (w_{col}) index in the weight matrices. Given the Step 3 base assignment $\Delta a = -\eta_g u_c$, the back-projection onto the weight matrices is simply

$$\boxed{\Delta\mathbf{W}^{\partial_x}_{i_{\text{loc}},\, w_{\text{col}}} \mathrel{+}= c_x \Delta a, \qquad \Delta\mathbf{W}^{\partial_y}_{i_{\text{loc}},\, w_{\text{col}}} \mathrel{+}= c_y \Delta a.} \tag{27}$$

This is `extract_neumann_sensitivities!`. It is gated by the same `active` mask and accumulates additively into the $\Delta\mathbf{W}$ already populated by the interior extraction.

5.4 Step 4 — backward through the RBF stencils

Given $\Delta\mathbf{W}^{\mathcal{L}}$ for each operator, we need $\Delta\boldsymbol{p}$ contributions such that $\partial\mathbf{W}^{\mathcal{L}}/\partial\boldsymbol{p} \cdot \Delta\boldsymbol{p}^{\top} = \Delta\mathbf{W}^{\mathcal{L}}$ in the standard pullback sense. This is provided by `_pullback_weights!` in `RadialBasisFunctions.jl`, which we call once per operator. The routine reuses the saddle-point factorization cached in Step 1 and runs purely as Julia linear algebra — no automatic differentiation framework is involved.

For each operator we call

$$\Delta\boldsymbol{p} \;\mathrel{+}=\; \texttt{\_pullback\_weights!}\big(\Delta\mathbf{W}^{\mathcal{L}},\, \mathbf{W}^{\mathcal{L}},\, \text{cache},\, \boldsymbol{p},\, \text{eval\_pts},\, \text{adjl},\, \text{basis},\, \mathcal{L}\big), \tag{28}$$

where `eval_pts` is the full point cloud for the interior operators and the Neumann sub-cloud for the Neumann operators (with an `eval_offset` array to map back to global indices for the Neumann case). The cost per operator is $O(Nk^3)$ for the per-row cache replay, dominated by the cube of stencil size.

5.5 Step 5 — external contributions

The three pieces collected in Step 5 are independent of the $\Delta\mathbf{W}$ pipeline and are added directly to $\Delta\boldsymbol{p}$.

5.5.1 Step 5a — the explicit $\partial\mathcal{L}/\partial\boldsymbol{p}$

The loss may depend on $\boldsymbol{p}$ directly through geometric quantities. The canonical case is the area penalty $\mathcal{L}_{\text{geom}}(\boldsymbol{p}) = \rho_{\text{pen}}(V(\boldsymbol{p}) - V_0)^2$. With the shoelace formula on the boundary loop, $\partial V/\partial\boldsymbol{p}_i$ is sparse (nonzero only on boundary points) and analytic. This term is added in the example via the helper `polygon_area_grad!`.

5.5.2 Step 5b — $\partial\boldsymbol{b}/\partial\boldsymbol{p}$ (Phase D Level 1)

When the traction $\boldsymbol{t}$ depends on $\boldsymbol{p}$ — the *dead-load* case — the contribution $\boldsymbol{\eta}^\top \partial\boldsymbol{b}/\partial\boldsymbol{p}$ is non-zero. The dependence can take two qualitatively different forms, and the machinery is set up to handle both:

Direct local dependence.
The traction at Neumann point g is a function only of $\boldsymbol{p}_g$ itself: $\boldsymbol{t}_g = \boldsymbol{t}(\boldsymbol{p}_g)$. A typical case is a profile prescribed by location, e.g. a parabolic shear $t_y(y) = P(D^2 - y^2)/(2I)$ depending on the local y-coord. The Jacobian $\mathbf{J}^{(i_{\text{loc}})} \equiv \partial\boldsymbol{t}_g/\partial\boldsymbol{p}_g$ is a single 2×2 matrix per Neumann point.

Indirect non-local dependence.
The traction at Neumann point g depends not only on $\boldsymbol{p}_g$ but on the coordinates of *other* boundary points through an intermediate geometric quantity. For example, in the cantilever the same parabolic-shear formula with a physically-meaningful effective half-depth $D_{\text{eff}} = (y_{\text{top corner}} - y_{\text{bot corner}})/2$ induces dependence of every right-edge Neumann traction on the two corner y-coordinates via $\partial\boldsymbol{t}/\partial D_{\text{eff}} \cdot \partial D_{\text{eff}}/\partial\boldsymbol{p}_{\text{corner}}$. The Jacobian $\mathbf{J}^{(i_{\text{loc}},q)} \equiv \partial\boldsymbol{t}_g/\partial\boldsymbol{p}_q$ is now in general non-zero for several $q \neq g$.

In both cases the gradient contribution is the same chain-rule product; the only difference is how many destinations the gradient flows to:

$$\boxed{\Delta\boldsymbol{p}_q \;\mathrel{+}=\; \begin{pmatrix}\eta_g\\ \eta_{g+N}\end{pmatrix}^{\top} \mathbf{J}^{(i_{\text{loc}},q)}, \qquad \mathbf{J}^{(i_{\text{loc}},q)}_{a,b} = \frac{\partial t_a}{\partial p_{q,b}}.} \tag{29}$$

The direct case is handled by `extract_load_sensitivities!` called with a per-Neumann-point 2×2 Jacobian. The indirect case is handled by a thin companion routine that, for each Neumann point whose traction depends on a non-local quantity $Q(\boldsymbol{p})$, contracts $(\partial t/\partial Q)$ with η_g, η_{g+N} and distributes the result through the (typically sparse) Jacobian $\partial Q/\partial\boldsymbol{p}$. The D_{eff} case above is one example; a follower pressure $\boldsymbol{t} = -p\,\boldsymbol{n}$ would be another, with the gradient flowing through both the polyline-normal Jacobian of Section **??** and a $\partial p/\partial\boldsymbol{p}$ if the pressure itself is shape-coupled.

Why this is not the only b-side contribution. The compliance loss $\mathcal{L} = \boldsymbol{b}^\top \boldsymbol{u}$ depends on $\boldsymbol{b}$ *explicitly*: $\partial\mathcal{L}/\partial\boldsymbol{b} = \boldsymbol{u}^\top$. The total b-side sensitivity is therefore

$$\left.\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{p}}\right|_b \;=\; \boldsymbol{\eta}^\top \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} \;+\; \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}}\,\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} \;=\; (\boldsymbol{\eta} + \boldsymbol{u})^\top \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{p}} \tag{30}$$

for compliance. The first term is the $\boldsymbol{\eta}$-side contribution and is computed inside `shape_gradient` when the traction Jacobian is supplied. The second is the $\boldsymbol{u}$-side contribution and must be added by the caller, by a *second* call to the same load-sensitivity routine with $\boldsymbol{u}$ in place of $\boldsymbol{\eta}$. Missing the second term introduces a $\sim 40\%$ FD error on a compliance loss — a hazard explicitly flagged in `CLAUDE.md`.

A note on the adjoint output. For losses with non-trivial $\partial\mathcal{L}/\partial\boldsymbol{b}$, the caller needs $\boldsymbol{\eta}$ *and* $\boldsymbol{u}$ available outside `shape_gradient` so it can run the second load-sensitivity contraction. The pipeline therefore returns $\boldsymbol{\eta}$ alongside $\boldsymbol{u}$ as part of the `shape_gradient` result, avoiding a redundant adjoint solve at the caller.

5.5.3 Step 5c — $\partial\boldsymbol{n}/\partial\boldsymbol{p}$ (Phase D Level 2)

This is the term that closes the differentiability of the Neumann assembly when the normals are shape-dependent. We give it a dedicated section (Section **??**) because the derivation is substantial.

6 Differentiable Neumann data

6.1 Phase D Level 1 — shape-dependent traction values

The case where $\boldsymbol{t}$ depends on $\boldsymbol{p}$ was covered in Section **??**. The implementation surface is small — a kwarg `traction_jacobians` on `shape_gradient`, plus the helper `extract_load_sensitivities!` — because the analytic content (the per-point 2×2 $\partial\boldsymbol{t}/\partial\boldsymbol{p}_g$ matrix) is supplied by the caller, and the framework just does the contraction with $\boldsymbol{\eta}$ (and, if needed, with $\boldsymbol{u}$). The framework is backward-compatible: omitting `traction_jacobians` reduces to the frozen-load case used by earlier phases.

6.2 Phase D Level 2 — differentiable normals

6.2.1 Motivation

The normals $\boldsymbol{n}$ enter the Neumann row coefficients in (**??**). If $\boldsymbol{n}$ is frozen at a reference value, $\partial\mathbf{A}/\partial\boldsymbol{n} \cdot \partial\boldsymbol{n}/\partial\boldsymbol{p} = 0$ and the Neumann assembly contributes to the gradient only through $\partial\mathbf{W}^{\partial_x}/\partial\boldsymbol{p}$ and $\partial\mathbf{W}^{\partial_y}/\partial\boldsymbol{p}$. This is sufficient when boundary points are constrained to move along their reference edge (so their normals do not rotate), but fails for any geometry where boundary points can move freely — in particular, for corner DOFs and for free-form boundaries.

6.2.2 Chord-formula vertex normal

Let $\mathcal{P} = (P_1, P_2, \ldots, P_M)$ be the CCW-ordered closed polyline of boundary vertices, with cyclic indexing. For Neumann point i_{loc} at position k in $\mathcal{P}$ we define

$$\boldsymbol{n}_{i_{\mathrm{loc}}}(\boldsymbol{p}) \;=\; \mathbf{R}(-\pi/2)\,\frac{\boldsymbol{p}_{P_{k+1}} - \boldsymbol{p}_{P_{k-1}}}{\left\|\boldsymbol{p}_{P_{k+1}} - \boldsymbol{p}_{P_{k-1}}\right\|}, \qquad \mathbf{R}(-\pi/2) = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}. \tag{31}$$

At an edge midpoint where $P_{k\pm1}$ are collinear with P_k this reduces to the canonical edge normal; at a polygon corner it yields a length-weighted bisector, biased toward the longer of the two adjacent edges. We discuss the corner subtlety in Section **??**.

6.2.3 Closed-form sparse Jacobian

With $\boldsymbol{d} \equiv \boldsymbol{p}_{P_{k+1}} - \boldsymbol{p}_{P_{k-1}}$, $\ell = \|\boldsymbol{d}\|$, and unit tangent $\boldsymbol{t} = \boldsymbol{d}/\ell = (t_x, t_y)$, direct differentiation of (**??**) gives

$$\frac{\partial \boldsymbol{n}}{\partial \boldsymbol{d}} \;=\; \frac{1}{\ell}\begin{pmatrix} -t_x t_y & t_x^2 \\ -t_y^2 & t_x t_y \end{pmatrix}, \qquad \frac{\partial \boldsymbol{d}}{\partial \boldsymbol{p}_{P_{k+1}}} = +\mathbf{I}, \quad \frac{\partial \boldsymbol{d}}{\partial \boldsymbol{p}_{P_{k-1}}} = -\mathbf{I}. \tag{32}$$

The Jacobian of $\boldsymbol{n}_{i_{\text{loc}}}$ with respect to all boundary coordinates is sparse with nonzeros at exactly two vertices, P_{k-1} and P_{k+1}. We package this in a `NormalJacobian` struct storing the four $\mathbb{R}^2$ blocks $\partial n_a/\partial \boldsymbol{p}_{P_{k\pm1}}$ for $a \in \{x, y\}$. A standalone unit test (`examples/test_polyline_normals.jl`) verifies (**??**) against `FiniteDifferences.central_fdm(5,1)` with worst-case relative error 3.9×10^{-12} on a perturbed rectangle.

6.2.4 Adjoint contribution from $\partial \boldsymbol{n}/\partial \boldsymbol{p}$

Returning to (**??**), the L2 contribution is $-\boldsymbol{\eta}^\top(\partial \mathbf{A}/\partial \boldsymbol{n})(\partial \boldsymbol{n}/\partial \boldsymbol{p})\boldsymbol{u}$. Each Neumann row g contributes

$$-\eta_g \sum_c \frac{\partial \mathbf{A}[g,c]}{\partial \boldsymbol{n}} \frac{\partial \boldsymbol{n}_{i_{\text{loc}}}}{\partial \boldsymbol{p}_q}\, u_c. \tag{33}$$

Because each Neumann row entry (**??**) is a linear function of (n_x, n_y), the partial derivatives $\partial \mathbf{A}[g,c]/\partial n_a$ are constants drawn from $\{0, \lambda^\star, \mu, \lambda^\star{+}2\mu\}$ multiplied by either $\mathbf{W}^{\partial_x}_{i_{\text{loc}},c'}$ or $\mathbf{W}^{\partial_y}_{i_{\text{loc}},c'}$. Pre-contracting over c yields scalar coefficients $r_a^{(g)}$:

$$r_a^{(g)} \;\equiv \sum_{p \in \text{entries}(g)} \frac{\partial \alpha_p}{\partial n_a}\, \mathbf{W}^{\mathcal{L}_p}[i_{\text{loc}}, w_p]\, u_{c_p}, \qquad a \in \{x, y\}. \tag{34}$$

The L2 contribution to $\Delta \boldsymbol{p}_q$ is then

$$\boxed{\Delta \boldsymbol{p}_q \;\mathrel{+}=\; -\sum_{a \in \{x,y\}} \left(\eta_g\, r_a^{(g)} + \eta_{g+N}\, r_a^{(g+N)}\right) \frac{\partial n_a}{\partial \boldsymbol{p}_q},} \tag{35}$$

nonzero only for $q \in \{P_{k-1}, P_{k+1}\}$ via the sparse `NormalJacobian`. This is the routine `extract_normal_sensiti` It is invoked from `shape_gradient` when the caller supplies the `normal_jacobians` kwarg; default `nothing` preserves the original frozen-normal behavior.

6.2.5 The corner pathology

For a 90° corner with adjacent edge segments of lengths $\Delta x \neq \Delta y$, the chord normal (**??**) is *not* the angular bisector of the two adjacent edge normals. At the cantilever's top-right corner with $\Delta x = 1.0, \Delta y = 0.5$, the chord formula gives $\boldsymbol{n} = (1, 2)/\sqrt{5} \approx (0.447, 0.894)$, biased toward the top-edge normal. Imposing $\boldsymbol{\sigma}\cdot\boldsymbol{n} = \mathbf{0}$ at the corner with this normal gives the linear combination $\sigma_{xx} + 2\sigma_{xy} = 0$ and $\sigma_{xy} + 2\sigma_{yy} = 0$, which is misaligned with the beam-natural condition $\sigma_{xx} = 0$ at the free end.

In the cantilever example we resolve this pragmatically by overriding the corner normal to the canonical $(1, 0)$ and zeroing the corresponding `NormalJacobian` entries. The corner *coordinates* remain free design variables; the L2 sensitivity still flows through the corner because the corner appears as one of the (P_{k-1}, P_{k+1}) neighbors of its adjacent edge midpoints, and those midpoints' chord normals rotate when the corner moves. The general handling of corners — selecting between the chord formula, the angular bisector, or a physics-aware convention — belongs in the geometry kernel and is discussed in Section **??**.

7 Sensitivity filtering on the boundary loop

7.1 Why filter at all?

The raw discrete gradient $\Delta \boldsymbol{p}^{\text{raw}} \equiv \mathrm{d}\mathcal{L}/\,\mathrm{d}\boldsymbol{p}$ produced by Steps 1–5 is the exact gradient of the discrete loss function. It is also typically contaminated, in RBF-FD discretizations, by a *Nyquist-mode* noise component: along each smooth edge of the boundary, the gradient values at adjacent boundary nodes can have large alternating-sign differences. This is not a bug — the FD validation passes to 10^{-7} — but it is an artefact of the local RBF stencil truncation. Without filtering, gradient descent amplifies this mode at every iteration and the design quickly diverges into a checkerboard pattern.

7.2 The 3-point Nyquist filter

The simplest mitigation is a single-pass 3-point average along the CCW boundary loop:

$$g_k^{\text{smooth}} \;=\; \alpha\, g_{k-1} + (1-2\alpha)\, g_k + \alpha\, g_{k+1}, \qquad \alpha = \tfrac{1}{4}. \tag{36}$$

The transfer function for a periodic sinusoid of wavenumber ω per segment is

$$H_3(\omega) \;=\; 1 - 4\alpha \sin^2(\omega/2) \;=\; \cos^2(\omega/2) \quad \text{at } \alpha = \tfrac{1}{4}. \tag{37}$$

This is a strict zero at $\omega = \pi$ (Nyquist mode killed) but leaves intermediate frequencies largely intact: $H_3(\pi/2) = \frac{1}{2}$, $H_3(\pi/4) \approx 0.85$. The optimizer can therefore still pump energy into modes around half-Nyquist and below.

7.3 The Helmholtz-PDE filter

To suppress medium-wavelength modes we solve, instead, an elliptic PDE on the closed boundary loop:

$$(\mathbf{I} - r^2\, \nabla^2_{\text{loop}})\, \boldsymbol{g}^{\text{smooth}} \;=\; \boldsymbol{g}^{\text{raw}}, \tag{38}$$

where ∇^2_{loop} is the standard 1D periodic Laplacian on the loop, stencil $[-1, 2, -1]$ at each vertex with its cyclic neighbors. The parameter r has units of boundary segments and sets the length scale of attenuation. Frozen boundary entries are imposed as identity rows in the system so the gradient is attenuated as it approaches a clamped (Dirichlet) region.

The Fourier-mode multiplier is

$$H_H(\omega; r) \;=\; \frac{1}{1 + 4r^2 \sin^2(\omega/2)}. \tag{39}$$

Selected values:

Table 1: Filter transfer at representative wavenumbers. H_3 is the 3-point average; $H_H(\cdot; r)$ is the Helmholtz filter (**??**).

ω	H_3	$H_H(r=1)$	$H_H(r=2)$
0 (DC)	1.00	1.00	1.00
$\pi/4$ (period 8)	0.85	0.62	0.29
$\pi/2$ (period 4)	0.50	0.33	0.09
π (Nyquist)	0.00	0.20	0.06

7.4 Implementation

The discrete system is a cyclic tridiagonal of size M (number of boundary loop vertices); for $M \sim$ 24 a dense $M \times M$ solve is microseconds. The implementation in `examples/shape_optimization_phase3_canti` is:

Listing 1: Helmholtz boundary filter — discrete cyclic Laplacian with frozen-entry rows.

```
function helmholtz_filter_along_boundary!(out, in_, loop, free_mask; r=1.0)
    out .= in_
    n_loop = length(loop)
    A = zeros(n_loop, n_loop)
    rhs = zeros(n_loop)
    for k in 1:n_loop
        i_curr = loop[k]
        rhs[k] = in_[2 * i_curr]
        if free_mask[2 * i_curr]
            k_prev = mod1(k - 1, n_loop); k_next = mod1(k + 1, n_loop)
            A[k, k_prev] = -r*r;  A[k, k] = 1 + 2*r*r;  A[k, k_next] = -r*r
        else
            A[k, k] = 1.0  # identity row, frozen entry passes through
        end
    end
    f = A \ rhs
    for k in 1:n_loop
        i_curr = loop[k]
        free_mask[2 * i_curr] || continue
        out[2 * i_curr] = f[k]
    end
    return out
end
```

For larger boundary loops the dense solve would be replaced by a sparse cyclic-tridiagonal solver (Thomas algorithm + Sherman–Morrison), which is $O(M)$.

8 Verification methodology

The discrete-adjoint pipeline is checked at every phase by comparing the analytic gradient to a fifth-order central finite-difference reference. The validation is built into each of the standalone Phase A and Phase B examples and into the illustrative Phase 3 cantilever:

$$\text{rel\_err}(\boldsymbol{p}) \;\equiv\; \frac{\left\| \nabla_{\boldsymbol{p}} \mathcal{L} \big|_{\text{AD}} - \nabla_{\boldsymbol{p}} \mathcal{L} \big|_{\text{FD}} \right\|_2}{\left\| \nabla_{\boldsymbol{p}} \mathcal{L} \big|_{\text{FD}} \right\|_2}, \tag{40}$$

with the FD reference computed by `FiniteDifferences.central_fdm(5,1)`. The targets are machine precision: rel_err $\sim 10^{-11}$ for Dirichlet-only problems (Phase A), $\sim 10^{-7}$ for problems with traction BCs (Phase B and beyond — the lower precision is due to FD truncation error on the larger relevant scale, not to the AD pipeline).

For the cantilever in Section **??** we additionally probe two qualitative properties:

- **Top/bottom symmetry.** The cantilever geometry and loading are symmetric under $y \to -y$. We match top-edge nodes to their bottom-edge mirrors by x-coordinate and compute $\max_i |g_y(\text{top}_i) + g_y(\text{bot}_i)|$. On the reference geometry this is $\sim 10^{-8}$, i.e. symmetric to floating-point rounding.

- **Spectral content of the raw gradient.** We project the iter-0 gradient onto the alternating-sign mode ϕ_π of the boundary loop and check its weight. Globally this is

essentially zero ($\sim 10^{-9}$); on each individual edge, however, the raw gradient is strongly Nyquist-aligned, by an amount the 3-point filter is designed to suppress.

Together these three diagnostics — finite differences, symmetry, and spectral content — are sufficient to distinguish a true implementation error (which would fail finite-differences) from a regularization issue (which would fail spectral content but pass finite differences) from a discretization symmetry break (which would fail symmetry alone).

9 Illustrative example: the cantilever

9.1 Setup

We illustrate the pipeline end-to-end on a 2D cantilever. Domain $[0, L] \times [-D, D]$ with $L = 8.0$, $D = 1.0$, modulus $E = 10^7$, Poisson ratio $\nu = 0.3$, RBF basis $\phi(r) = r^3$ with augmenting polynomial degree 3 and stencil size $k = 35$. We discretize with a 9×5 regular grid ($N = 45$). The loading is a parabolic shear traction on the right edge, $t_y(y) = P(D^2 - y^2)/(2I)$ with $P = 10^3$, $I = 2D^3/3$, zero on top and bottom. The left edge is clamped.

The loss is compliance plus a two-sided area penalty:

$$\mathcal{L}(\boldsymbol{p}) = \boldsymbol{b}(\boldsymbol{p})^\top \boldsymbol{u}(\boldsymbol{p}) \; + \; \rho_{\text{pen}} \big(V(\boldsymbol{p}) - V_0\big)^2, \qquad \rho_{\text{pen}} = 10^3. \tag{41}$$

Design variables are the y-coordinates of the Neumann-boundary points; the optimizer is plain gradient descent with Armijo backtracking; the iteration budget is 40.

9.2 End-to-end workflow

At each iteration:

1. Refresh the live traction values and (when L2 is active) the live polyline normals on `traction_layout`.
2. Compute the analytic gradient via `shape_gradient` (Steps 1–4 + the Phase D L1 $\boldsymbol{\eta}$-side b-contribution).
3. Add the $\boldsymbol{u}$-side Phase D L1 contribution (`extract_load_sensitivities!` with $\boldsymbol{u}$).
4. Add the area-penalty gradient (Step 5a).
5. Apply the sensitivity filter on the boundary loop, then mask to the free DOFs.
6. Armijo backtracking line search; update $\boldsymbol{p}$.

9.3 Snapshot results

Numerical results are reported here as a snapshot of the pipeline's qualitative behavior during active development. The *absolute* compliance figures may shift as the cantilever example is refined — in particular, an artefact in the tip-shrinkage gradient sign was masked by an earlier corner-freeze convention and is being repaired at the time of writing. The *qualitative* contrasts between configurations (effect of L2 on the corner DOF, effect of filter choice on the boundary smoothness) are robust and survive that refinement.

Table 2: Snapshot of three configurations on the cantilever benchmark, 40 iterations each. All configurations pass the finite-difference validation at iter 0 to rel_err $\sim 10^{-7}$.

Configuration	$\Delta C/C_0$	boundary appearance
(A) corners frozen, 3-point filter	-47.6%	mild waviness
(B) corners free, 3-point filter	-52.1%	visible long-wavelength undulation
(C) corners free, Helmholtz r=1 filter	-46.6%	visibly smooth

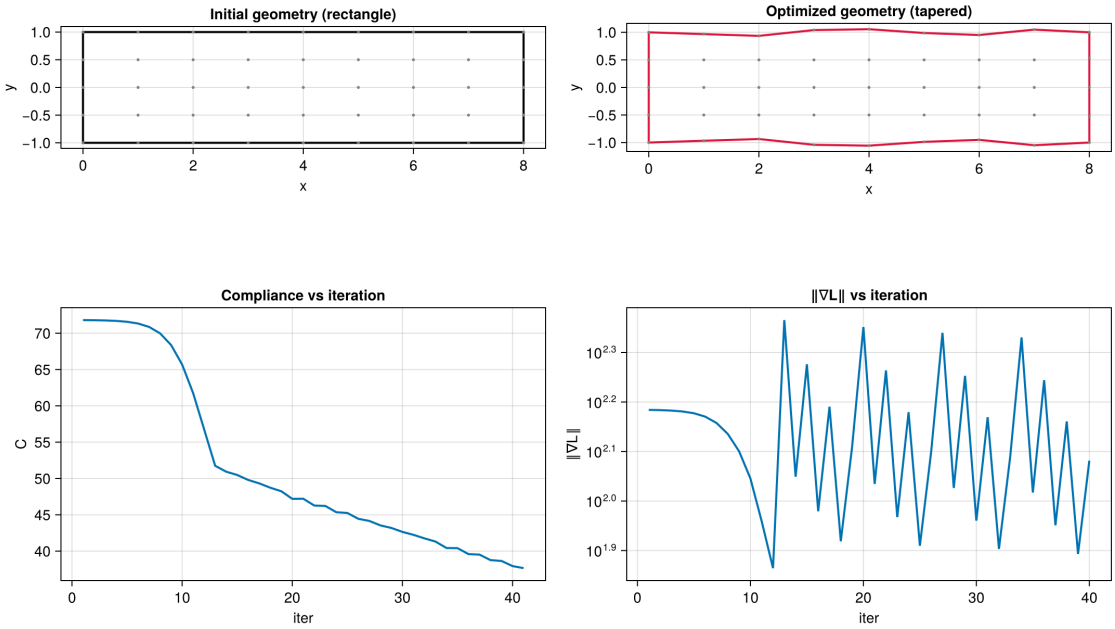

Figure 1: Configuration (A): corners frozen, 3-point filter.

9.4 What the three configurations demonstrate

Despite the noted ongoing refinement of the cantilever example, the contrast between (A)–(C) carries information that is robust:

(1) Going from (A) to (B) demonstrates that the Phase D L2 machinery is doing real algorithmic work: unfreezing the corner DOFs gives the optimizer more design freedom, and the gradient through the chord-formula normals is correct (FD-validated).

(2) Going from (B) to (C) demonstrates that the 3-point filter is insufficient for shape regularization in the presence of richer design freedom: the optimizer exploits unfiltered modes, and the Helmholtz filter is necessary to recover a smooth-design optimum.

(3) Configuration (C) is the appropriate baseline against which to compare future algorithmic improvements (adaptive r, augmented-Lagrangian area constraint, modern optimizer): the regularization is honest and the compliance number is not inflated by noise exploitation.

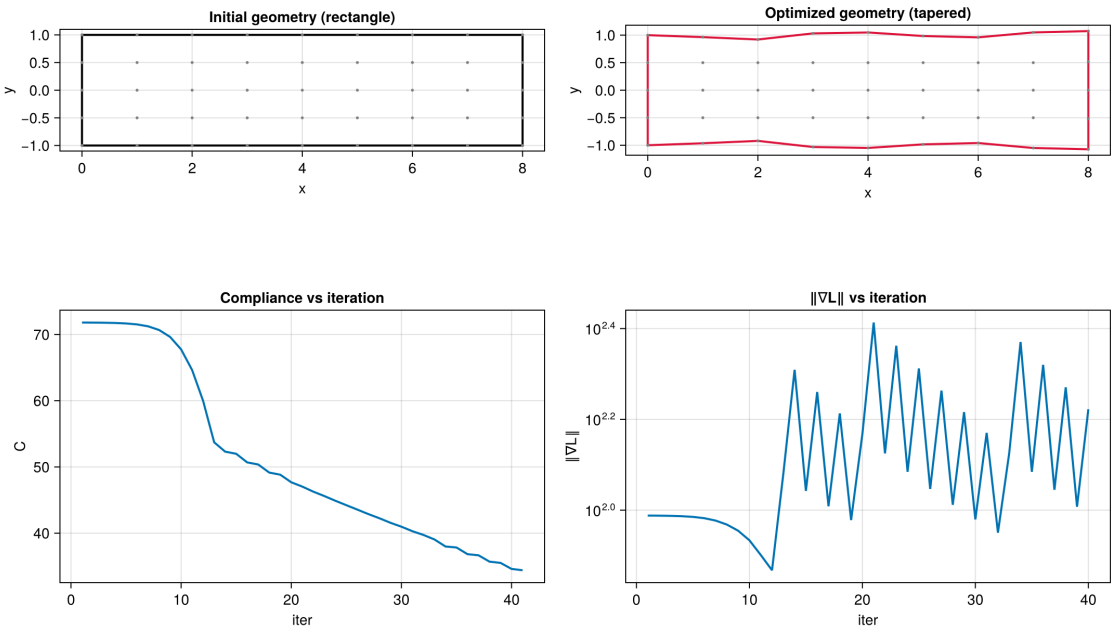

Figure 2: Configuration (B): corners free (Phase D L2 active), 3-point filter. The optimizer extracts additional compliance reduction by exploiting medium-wavelength modes the 3-point filter leaves unattenuated, producing visible boundary undulations.

10 The plate with a hole: from gradient filtering to a design-space and re-meshing architecture

The cantilever validated the adjoint but left a question the filter of Section **??** only papered over: *why* does the raw per-node gradient need filtering at all, and is filtering the right cure? The plate-with-hole problem — a square plate in biaxial tension with an elliptical hole, optimized toward the circular hole that minimizes compliance — answered both, and in doing so reorganized the optimizer around a design parameterization and an actively re-meshed cloud. This section records that turn; the granular record is the companion log `boundary_gradient_noise.md`.

10.1 The symptom and the diagnosis: $\partial \mathbf{W}/\partial \boldsymbol{p}$

On grids fine enough to resolve the hole, the per-node ℓ^2 shape gradient is dominated by a near-Nyquist sawtooth: the gradient spectrum *rises* toward the highest representable boundary mode, so a near-Nyquist wiggle carries $\sim 40\times$ the gradient of the circle-seeking $\cos 2\theta$ mode. The gradient is not wrong — it is finite-difference validated mode by mode to 10^{-6}, the near-Nyquist modes included — it is genuinely *rough*.

Fixed-cloud probes localized the mechanism. With the PDE removed and replaced by smooth analytic fields f, η, the surrogate $S = \eta^\top \mathbf{W} f$ differentiated with respect to a boundary node's radial position still produces a broadband, Nyquist-rich $\partial S/\partial r_j$. The roughness is therefore the *geometry sensitivity of the RBF-FD differentiation weights themselves*, $\partial \mathbf{W}/\partial \boldsymbol{p}$, independent of the physics (it reproduces on a thermal Laplace problem). It grows with the operator's derivative order (the q=2 elasticity/Laplacian operators are $\sim 46\times$ worse than the q=1 Neumann rows, because the pull-back of a second-order operator probes the third derivative of the r^3 kernel), worsens with kernel order (a conditioning effect), and sharpens under refinement. Crucially it is *broadband* and overlaps the physical signal in frequency.

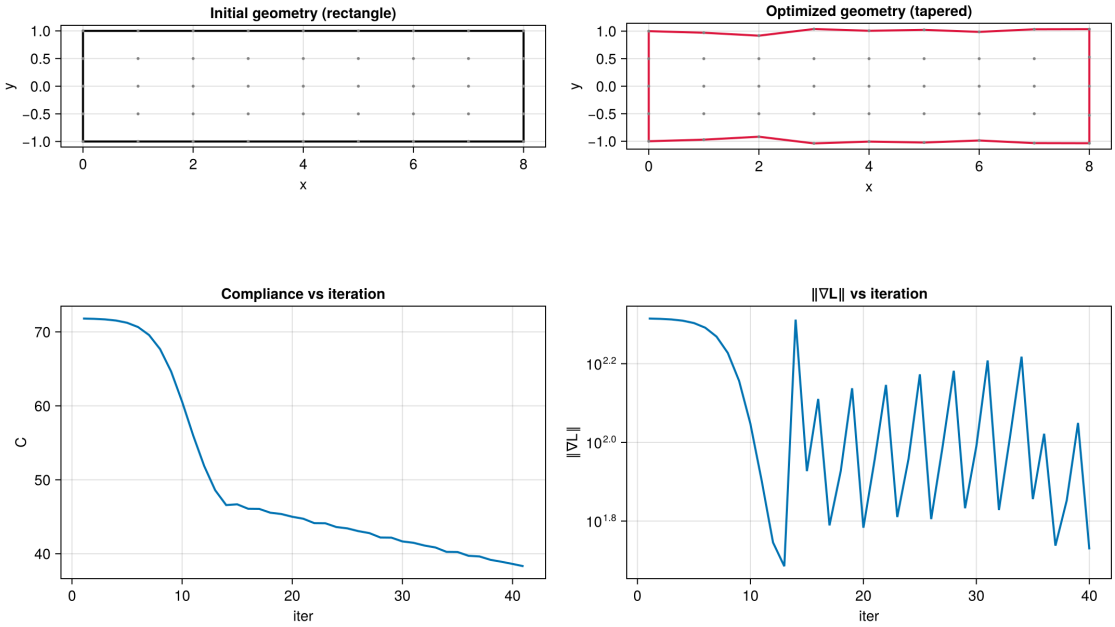

Figure 3: Configuration (C): corners free, Helmholtz r=1 filter. The boundary is now visibly smooth. The compliance reduction is slightly lower than (B) because the optimizer is restricted to the smooth design subspace; this is the honest optimum under physical regularization rather than the apparent optimum that exploits discretization noise.

10.2 Why filtering is the wrong cure

The broadband overlap is decisive: *no frequency-domain filter on the gradient can separate the artifact from the signal*, because they share frequencies. A Helmholtz/Sobolev/Tikhonov smoother does lower the high-to-low energy ratio, but a faithfulness check — the cosine of the smoothed gradient against the true low-mode gradient — shows it corrupts the very signal it is meant to preserve. The filter of Section **??** is accordingly demoted from "the cure" to "a mild conditioner usable only atop a structurally smooth design space."

The structural cure removes the rough degrees of freedom *by construction*. We stop treating the boundary nodes as the design and parameterize the hole radius by a truncated Fourier series

$$r(\theta) = r_0 + \sum_{m \in \mathcal{M}} \big(a_m \cos m\theta + b_m \sin m\theta\big), \tag{42}$$

with the area held fixed by solving for r_0. The noisy nodal gradient is *contracted* onto the design modes by the exact chain rule, $\partial C/\partial a_m = \sum_j g_{\mathrm{rad},j} \cos m\theta_j$, which integrates the per-node noise out. This is a Hilbertian (Riesz) gradient in a finite-dimensional smooth design space; the adjoint of Sections **??**–**??** is untouched — only the final contraction $\mathbf{J}^\top \boldsymbol{g}$ is new.

10.3 The clean-DOF budget and its auto-calibration

A design space helps only if its modes sit *below the artifact floor*. The admissible mode count is set by the cloud, not by hand. The artifact decorrelates at the stencil radius ρ (the largest neighbour distance in a stencil), so a boundary mode is trustworthy only if its half-wavelength spans at least one stencil radius — a stencil-Nyquist criterion $m \le \lfloor \pi r/\rho \rfloor$. The Sobolev length of the residual conditioner is likewise tied to the cloud, $\ell_{\mathrm{sob}} = \rho/r$. Both knobs are derived, not tuned:

knob	formula	value (dx=0.05, k=35)
stencil radius ρ	max neighbour distance	$0.30 = 6\,\mathrm{d}x$
mode cap m_{cap}	$\lfloor \pi r/\rho \rfloor$	2
Sobolev length	ρ/r	1.06

At this resolution the clean-DOF budget is a single mode — the ellipse mode. Hand-set richer truncations (m=2:4) are unstable: the optimizer random-walks the under-resolved m=3, 4 channels up until a boundary wiggle breaks a stencil. The calibrated cap closes them by construction; unlocking higher modes requires refinement (smaller ρ/r), a resolution/cost trade rather than a tuning knob.

10.4 Three failure modes, and why they need different cures

With the gradient exact and the design space clean, the binding obstacle changed identity. Three distinct failures, conflated in the earlier work, must be kept apart:

(A) Descent-direction noise. The $\partial \mathbf{W}/\partial \boldsymbol{p}$ roughness above. Intrinsic to the discrete gradient — present on a *fresh* cloud at iteration zero — and cured only by the design-space contraction.

(B) Cloud degradation. As nodes move, a cloud frozen at an old geometry ill-conditions: frozen stencils stretch, frozen adjacency goes stale. A stale-cloud artifact.

(C) Objective bias. A cloud fitted to a *stale* geometry has its discrete-compliance optimum displaced from the true one — on the plate, $a_2^\star \approx \pm 0.07$ rather than the true circle $a_2 = 0$, the sign bracketed between a frozen interior ($+0.078$) and a Laplace-morphed interior (-0.065). The shallow optimum (compliance varies $\sim$2% across the range) amplifies the $O(h^p)$ objective error into an $O(0.07)$ design error. Also a stale-cloud artifact.

That (C) is a stale-cloud artifact and not intrinsic was settled by a fresh-cloud test: at the true circle, a cloud freshly generated for that geometry has $\partial C/\partial a_2 \approx 5 \times 10^{-8}$, three orders below the driving gradient — the circle *is* the discrete optimum of an appropriately re-initialized cloud. Averaged over independent fresh clouds the residual m=2 gradient maps to $a_2^\star \approx 0.008$, a tenfold debiasing.

A practical corollary closes a tempting shortcut: the old per-node route could *not* have been rescued by periodic re-initialization alone. Re-meshing cures (B) and (C) but not (A); on a fresh cloud the per-node gradient is still Nyquist-dominated, so the boundary roughens immediately. Re-meshing helps the per-node route only if it *smooths* the boundary — which is the parameterization in disguise. (A) and (B, C) need different cures; the parameterization is not optional.

10.5 The two-front architecture

The cures compose into a single loop with two cooperating fronts on the cloud, the design space handling (A):

Front 1 — morph (within a re-mesh interval). The interior is slaved to the boundary by a smooth, differentiable Laplace extension, $\Delta \boldsymbol{p}_{\mathrm{int}} = \mathbf{M}\,\Delta \boldsymbol{p}_{\mathrm{bnd}}$ with $\mathbf{M} = -\mathbf{L}_{ii}^{-1}\mathbf{L}_{ib}$, factorized once per anchor. The adjoint flows through it by the transpose, $\hat{\boldsymbol{g}}_{\mathrm{bnd}} = \boldsymbol{g}_{\mathrm{bnd}} + \mathbf{M}^\top \boldsymbol{g}_{\mathrm{int}}$; omitting this term makes the boundary-only gradient $O(1)$ wrong once the interior moves. Fixed stencils within the interval keep the objective smooth.

Front 2 — re-anchored re-mesh (between intervals). When a quality indicator trips, a fresh high-quality cloud is generated for the current design and the morph re-anchored to it (reference reset, operator refactorized). Each re-mesh reads as a *re-initialization*, not a perturbed continuation — this is what removes (B) and (C). Re-meshing is not precluded by the discrete adjoint; it is precluded only if one differentiates *through* the re-mesh or demands exact stationarity of a single fixed discrete objective. We do neither — we differentiate the smooth morph within an interval and re-anchor between intervals.

Because the discrete compliance *value* jumps at each re-mesh (by an amount bounded by the discretization accuracy — noise, not signal), descent is on the gradient, $\|\mathbf{J}^\top \boldsymbol{g}\| \to 0$, never gated on C. A gradient-decaying "settling" step was tried and rejected: under a stale morph it lingers in the distorted regime and corrupts the gradient.

10.6 Closed-loop cloud quality: which indicators matter

Front 2 is triggered by a registry of cloud-quality indicators, each a cheap scalar measured every iteration, individually toggleable with its own threshold; only enabled indicators trigger a re-mesh. On the ellipse→circle deformation:

indicator	measures	verdict
`morph_drift`	max interior displacement / $\mathrm{d}x$	active trigger; conservative
`stencil_growth`	frozen-stencil radius / anchor radius	principled but slack (≤ 1.09)
`min_gap`	closest interior–hole / $\mathrm{d}x$	dormant
`spacing_cv`	interior nearest-neighbour CV	dormant
`boundary_cv`	hole-node spacing CV	dormant
`min_sep`	global minimum separation / $\mathrm{d}x$	dormant

The smooth Laplace morph is gentle: a $0.58\,\mathrm{d}x$ node drift stretches the worst stencil by only $\sim 9\%$. The displacement proxy `morph_drift` fires first and is therefore conservative; `stencil_growth`, tied directly to operator validity, is the more principled trigger and would let intervals run longer. The remaining indicators are insurance for harsher regimes (a growing hole would wake `min_gap`; a strongly non-circular target, `boundary_cv`). `stencil_growth` captures "old neighbours separate" but not "non-neighbours approach" (adjacency-membership staleness), which needs a current k-NN query and was negligible here.

10.7 Result

The two-front loop converges to the circle: $a_2 : 0.0986 \to 0.0154$ (true 0), $r_0 : 0.2741 \to 0.2826$ (true 0.2828), compliance $5.18 \times 10^{-5} \to 4.88 \times 10^{-5}$, with two `morph_drift`-triggered re-meshes over sixty iterations. It lands at the true optimum, not at either stale-cloud bias ($+0.078$ or -0.065): the design space removes the noise, the re-anchoring removes the bias, and the morph keeps the cloud differentiable in between.

11 Generalization

The five-step pipeline is structured so that each step is PDE-agnostic except where the PDE explicitly enters. We summarize what changes when the PDE does.

11.1 Scalar elliptic PDEs

For scalar problems (heat equation, Poisson, electrostatics, magnetostatics) the system is $N \times N$, not $2N \times 2N$. Steps 1, 2, 4 are essentially unchanged. Step 3 simplifies: the coefficient extraction in (**??**) reduces from the four-coefficient family to a single per-operator constant. The Neumann

row family becomes $\partial u/\partial \boldsymbol{n} = g$ or $\alpha u + \beta \partial u/\partial \boldsymbol{n} = g$ (Robin), with a one-coefficient analogue of (**??**).

11.2 Nonlinear PDEs (Newton iteration)

For nonlinear $\boldsymbol{F}(\boldsymbol{u}, \boldsymbol{p}) = \mathbf{0}$ solved by Newton, the adjoint is built at the *converged* Jacobian:

$$\mathbf{J}(\boldsymbol{u}^\star)^\top \boldsymbol{\eta} \;=\; \big(\partial \mathcal{L}/\partial \boldsymbol{u}\big)^\top \quad \text{at the converged iterate } \boldsymbol{u}^\star. \tag{43}$$

Steps 1, 2, 4 carry over identically. Step 3 changes: the coefficient extraction now has solution-dependent coefficients (the Jacobian entries involve $\boldsymbol{u}^\star$ itself), and the $\partial \mathbf{F}/\partial \boldsymbol{p}$ contracts both with state and with the operator weight matrices. The detailed treatment for incompressible Navier–Stokes is sketched in `plan_manual_adjoint.md`.

11.3 Time-dependent PDEs

Transient problems require a backward-in-time sweep of the adjoint:

$$-\frac{\mathrm{d}\boldsymbol{\eta}}{\mathrm{d}t} \;+\; \mathbf{J}^\top \boldsymbol{\eta} \;=\; \big(\partial \mathcal{L}/\partial \boldsymbol{u}\big)^\top \quad \text{from } t = T \text{ back to } t = 0. \tag{44}$$

The architectural cost is checkpointing (memory for storing forward states at intermediate times) but the per-step structure is the same as the steady case.

11.4 Multi-physics coupling

Coupling several physics on the same point cloud — the fundamental advantage of the meshless approach for this project — is block-structural in the algebra: the joint $\mathbf{A}$ has diagonal blocks for each physics and off-diagonal blocks for couplings, all built from the same RBF stencils. The Step 3 coefficient extraction generalizes to per-block coefficient families. The Step 4 backward through the weight matrices is unchanged. Strongly-coupled problems with a coupled forward Newton solve fall under the nonlinear-PDE case above.

11.5 3D

The five-step pipeline is dimension-agnostic in its *structure*; only the per-operator content changes. In 3D:

- the interior weight matrix family grows to $\{\partial_{xx}, \partial_{yy}, \partial_{zz}, \partial_{xy}, \partial_{yz}, \partial_{xz}\}$;
- the Neumann row family in (**??**) becomes a 3×3 analogue (with 3 normal components and 3 stress components);
- the polyline normal in (**??**) is replaced by a triangle-based vertex-normal-average (sparse on the three vertices of each STL face), with the analogous closed-form Jacobian;
- the boundary-loop Helmholtz filter is replaced by a Laplacian on the discrete surface mesh.

None of these changes requires new mathematical content. The content is the same; the book-keeping is more.

12 Position relative to the state of the art

12.1 Reference points

The mature reference points for shape optimization in contemporary engineering practice are FEM-based commercial codes (Tosca, OptiStruct, NX Topology, ANSYS Mechanical Topology), density-based topology optimization via SIMP/RAMP (Bendsøe & Sigmund, codified in the 88-line MATLAB code of Andreassen et al. 2011), CAD-parameterized shape optimization (ESP, Vorpal, OpenFOAM's continuous-adjoint solvers), and the level-set / phase-field methods of Wang et al. 2003 onward. The workhorse optimizer is the Method of Moving Asymptotes (Svanberg), with IPOPT, L-BFGS-B, and SQP rounding out the toolbox. Helmholtz-PDE sensitivity filtering for SIMP was introduced by Lazarov & Sigmund (Int. J. Numer. Methods Eng., 2011) — essentially the filter we use here, applied to a 1D boundary loop rather than to the design density field.

12.2 Honest comparison

The present work is best described as *methodologically sound on a niche discretization, not yet operationally competitive.* Table **??** compares dimensions where the gap is most evident.

Table 3: Honest comparison of the present pipeline against the state of the art.

Dimension	State of the art	This work
PDE discretization	FEM at 10^6 DOFs; AMR; multi-physics	RBF-FD at $N \sim 45$ in the example; single physics (lin. elasticity); 2D
Adjoint correctness check	FD at small scale, otherwise checkpointed AD	Central order-5 FD vs analytic at every config; $\sim 10^{-7}$
Boundary normal differentiability	implicit through CAD/FEM mesh	Explicit chord formula with sparse closed-form Jacobian (Section **??**)
Sensitivity filter	Helmholtz PDE on the design field (Lazarov–Sigmund 2011) or radial average; arc-length / surface	Helmholtz PDE on the 1D boundary loop, uniform segments (Section **??**)
Optimizer	MMA, IPOPT, L-BFGS-B, SLP, SQP	Steepest descent with Armijo backtracking
Constraint handling	Augmented Lagrangian, projected gradient, SQP	Quadratic penalty with hand-tuned ρ_{pen}
Geometry kernel	CAD/NURBS/splines or STL-vertex direct, auto-meshing	Boundary points are the design (no kernel yet); STL absent
Mesh quality under deformation	Re-mesh / ALE / FFD	Not needed (meshless)
Multi-material / topology change	SIMP density field, level set	Out of scope of this report; meshless naturally permits it
Scale (DOFs)	10^4 to 10^7, 3D	$\sim 10^2$, 2D

12.3 Where the work might claim novelty

A defensible claim would be along the lines of: *a complete manual-adjoint shape-optimization pipeline for RBF-FD linear elasticity, with closed-form analytic sensitivities at the weight-matrix and normal levels and a Helmholtz boundary-loop sensitivity filter, validated to* 10^{-7} *against finite differences and structured for clean extension to multi-physics on the same point cloud.* RBF-FD has been used extensively for forward PDE solving (Bayona, Flyer, Fornberg and collaborators), but its application to shape optimization with analytic discrete adjoints is sparse in the literature; most existing RBF-FD shape-opt work relies on black-box AD or continuous-adjoint formulations that ignore discretization error. The deliberate separation of geometry (`WhatsThePoint.jl`) from PDE machinery (`Macchiato.jl`) is also unusual and well-motivated for meshless contexts.

This is a methodologically clean starting point in an under-explored corner of the field. It is not state-of-the-art performance, scale, or feature coverage; we will return to the gap in Section **??**.

13 Open work

13.1 Within the elasticity pipeline

Stronger optimizer. Steepest descent with Armijo converges slowly on anisotropic Hessians. The natural next step is L-BFGS-B for the box-constrained version of the problem and MMA for the full multi-constraint version. Both are wrapper-level work — the adjoint pipeline is unchanged.

Augmented Lagrangian for the area constraint. The current quadratic penalty produces a sawtooth in $\|\nabla\mathcal{L}\|$ as $V(\boldsymbol{p})$ oscillates around V_0 across line-search trials. An augmented Lagrangian, with multiplier updated between outer iterations, eliminates this artefact.

Adaptive Helmholtz r. The current value $r = 1$ is hand-tuned. A standard recipe in topology optimization is to start with a large r (aggressive smoothing during the early shape-finding phase) and reduce it toward zero as the design stabilizes. Implementing this is straightforward; the right schedule is empirical.

Arc-length-weighted boundary Laplacian. The current Helmholtz filter treats the boundary loop as a uniform 1D mesh, slightly over-smoothing longer segments relative to shorter ones. A geometric Laplacian with arc-length weights would be more rigorous; at the cantilever scale the visual impact is small.

Grid resolution study. $N = 45$ is too coarse to express a clean Michell taper. A scale test at $N \gtrsim 100$ would clarify whether the regularized optimum converges to a clean tapered beam as the grid refines.

13.2 Within the cantilever example

Tip-shrinkage gradient artefact. The cantilever example is currently being repaired: the gradient driving tip-shrinkage was found to have an incorrect sign component, which had been masked by an earlier corner-freeze convention. The repair is purely at the example level (loss definition / boundary parameterization) and does not affect the adjoint pipeline. Once landed, the quantitative numbers in Section **??** will be re-validated.

Reference textbook problem. We rely on FD-vs-AD consistency as the validation criterion. A problem with an analytic solution (Schade plate, thin-walled pressure vessel, Lamé's problem) would let us check the discretization itself — not just the adjoint.

13.3 Toward engineering geometries

Geometry kernel. Today the boundary points *are* the design variables. For real STL-driven geometries the differentiable chain $\boldsymbol{l} \to \mathcal{G}_{\text{STL}} \to \boldsymbol{p}_b$ is needed, with $\boldsymbol{l}$ either (i) STL vertices direct, (ii) spline / NURBS control points, or (iii) FFD lattice control points. This is the single gating item between the present R&D state and engineering applicability.

STL-side primitives in `WhatsThePoint.jl`. The cross-repo separation places STL operations, point cloud generation, and triangle-based vertex normals on the WTP side. Once the geometry kernel choice is made, the WTP work is: STL vertex perturbation, triangle-normal derivation with the analogue of (**??**), STL-to-pointcloud sampling with Jacobian, optional spline-to-STL retessellation.

Boundary-to-interior deformation. When boundary points move, interior collocation points must follow to maintain point-cloud quality. An RBF-interpolation approach that reuses the existing `_pullback_weights!` machinery is the natural choice and was already sketched in `plan_shape_optimization_pipeline.md`.

Scale-up. Forward and adjoint solves at $N \sim 10^4$–10^5 require iterative or preconditioned linear solvers tuned for RBF-FD elasticity. No off-the-shelf preconditioner exists; building one is a research-grade subproblem.

13.4 Beyond elasticity

Other physics, same pipeline. The five-step pipeline as derived in Section **??** is reusable for any PDE that fits the $\mathbf{A}(\boldsymbol{p})\boldsymbol{u} = \boldsymbol{b}(\boldsymbol{p})$ template, with adapted coefficient extraction in Step 3. Heat conduction, electrostatics, magnetostatics, and incompressible Stokes are direct extensions; incompressible Navier–Stokes requires the Newton-converged Jacobian variant (Section **??**).

Multi-physics shape optimization. The architectural advantage of meshless is that the same point cloud supports all of these physics with the same weight matrix family. Coupling is block-structural; the adjoint of the coupled system is built by the same Step 2 solve, with the joint right- hand side from each physics' contribution to the loss. The strategic motivation for the framework, articulated in `Vision.md`, is precisely this multi-physics tractability.

14 Conclusion

We have presented the complete discrete adjoint pipeline implemented in `Macchiato.jl` for RBF-FD shape optimization of plane-stress linear elasticity. Starting from the continuous shape-optimization problem, we derived the five-step decomposition in which the sensitivity calculation reduces to: a forward solve that caches the RBF stencils; a single adjoint solve; an analytic back-projection from $\Delta\mathbf{A} = -\boldsymbol{\eta}\boldsymbol{u}^\top$ onto per-operator sensitivities $\Delta\mathbf{W}^{\mathcal{L}}$ via the elasticity coefficient structure (interior) and the Neumann coefficient family (boundary); a pure-Julia backward pass through the RBF stencils via `_pullback_weights!`; and a closed set of explicit external contributions covering the area constraint, the shape-dependent traction values (Phase D Level 1), and the differentiable polyline normals (Phase D Level 2). The whole chain is validated to $\sim 10^{-7}$ relative error against fifth-order central finite differences.

The cantilever motivated a sensitivity filter on the boundary loop, but the plate-with-hole study of Section **??** showed that frequency filtering is the wrong cure: the boundary gradient noise is the geometry sensitivity of the RBF-FD weights, $\partial \mathbf{W}/\partial \boldsymbol{p}$, which is broadband and overlaps the signal, so no gradient filter separates them faithfully. The cure is structural — a clean-DOF Fourier design space (calibrated to the stencil-resolved mode budget) that removes the rough degrees of freedom by construction, an actively re-anchored re-mesh that removes the stale-cloud objective bias, and a differentiable Laplace morph that keeps the cloud smooth in between. On the plate the resulting two-front optimizer converges to the true circular optimum, landing at neither stale-cloud bias; the filter survives only as a mild conditioner atop the smooth design space.

The framework is structured for clean extension to other physics on the same point cloud, to three dimensions with vertex-normal gradients on STL triangles, and to multi-physics shape optimization — the strategic target of the broader project. Each ingredient is in place; what remains is engineering extension rather than mathematical invention. The path from this working document to a publishable paper runs through the geometry kernel decision and a representative engineering test case; those are the next milestones, and the present pipeline is the foundation they will rest on.